

Correspondence between NUTS implemented by hand and the Gellmann's article

Tommaso Pirazzo

In the following there is the 6th algorithm of the article "*The No-U-Turn Sampler: Adaptively Setting Path Lengths in Hamiltonian Monte Carlo*" with the indicated correspondences with the code implemented in the file `JAX_NUTS_functions.ipynb`.

1 Leapfrog integrator

2 FindReasonableEpsilon

Algorithm 1: Heuristic for choosing an initial value of ϵ

Input: Current position θ
Output: Initial step size ϵ
Sample momentum $r \sim \mathcal{N}(0, I)$;
 $\epsilon \leftarrow 1$;
 $E_0 \leftarrow \log p(\theta, r)$;
 $(\theta', r') \leftarrow \text{Leapfrog}(\theta, r, \epsilon)$;
 $E_1 \leftarrow \log p(\theta', r')$;
 $\log_accept \leftarrow E_1 - E_0$;
if $\log_accept > \log(0.5)$ **then**
 $a \leftarrow +1$;
else
 $a \leftarrow -1$;
while $a \cdot \log_accept > -a \cdot \log(2)$ **do**
 $\epsilon \leftarrow \epsilon \cdot 2^a$;
 $(\theta', r') \leftarrow \text{Leapfrog}(\theta, r, \epsilon)$;
 $E_1 \leftarrow \log p(\theta', r')$;
 $\log_accept \leftarrow E_1 - E_0$;
return ϵ ;

```
1 def FindReasonableEpsilon(theta, key) :
2     assert theta.ndim == 1
3
4     # sample momentum r ~ N(0, I)
5     key, subkey = jrnd.split(key)
6     r = jrnd.normal(subkey, shape=theta.shape)
7
8     # initial epsilon
9     epsilon = 1.0
10
11    # compute initial log joint density
12    E0 = log_p(theta, r)
13
14    # one leapfrog step
15    theta_prime, r_prime = Leapfrog(theta, r, epsilon)
16    E1 = log_p(theta_prime, r_prime)
17
```

```

18 # log acceptance probability
19 log_accept = E1 - E0
20
21 # direction: +1 (increase) or -1 (decrease)
22 a = jnp.where(log_accept > jnp.log(0.5), 1.0, -1.0)
23
24 # -----
25 # cond_fun: while loop condition
26 # -----
27 def cond_fun(state):
28     eps, log_acc = state
29     return (a * log_acc) > (-a * jnp.log(2.0))
30
31 # -----
32 # body_fun: loop body
33 # -----
34 def body_fun(state):
35     eps, _ = state
36     eps = eps * (2.0 ** a)
37     theta_p, r_p = Leapfrog(theta, r, eps)
38     E1 = log_p(theta_p, r_p)
39     log_acc = E1 - E0
40     return (eps, log_acc)
41
42 # while loop in JAX
43 epsilon, _ = jax.lax.while_loop(cond_fun, body_fun, (
44     epsilon, log_accept))
45
46 return epsilon

```

3 BuildTree

This pseudo-code is referred to the **BuildTree** function implemented in the **Algorithm 6** (No-U-Turn Sampler with Dual Averaging) of the article. This function is composed by two cases:

- $j = 0$: Base case $\rightarrow$ take one leapfrog step in the direction v .
- $j \neq 0$: Recursion $\rightarrow$ implicitly build the left and right subtrees.

The "base case" is implemented before in the function `build_tree_base` and then called in the complete function `BuildTree`.

Algorithm 2: BuildTree($\theta, r, u, v, j, \epsilon$)

Input: Position θ , momentum r , slice variable u , direction

$v \in \{-1, +1\}$, depth j , step size ϵ

Output: (θ^-, r^-) , (θ^+, r^+) , candidate θ' , number of valid points n' ,
stop flag s' , acceptance sum α' , acceptance count n'_α

if $j = 0$ **then**

$(\theta', r') \leftarrow \text{Leapfrog}(\theta, r, v \cdot \epsilon);$

$E' \leftarrow \log p(\theta', r');$

$n' \leftarrow \mathbb{I}[u < \exp(E')];$

$s' \leftarrow \mathbb{I}[u < \exp(E')];$

$\Delta \leftarrow E' - \log p(\theta, r);$

$\alpha' \leftarrow \min\{1, \exp(\Delta)\};$

$n'_\alpha \leftarrow 1;$

return $(\theta', r'), (\theta', r'), \theta', n', s', \alpha', n'_\alpha;$

$(\theta^-, r^-), (\theta^+, r^+), \theta', n', s', \alpha', n'_\alpha \leftarrow \text{BuildTree}(\theta, r, u, v, j - 1, \epsilon);$

if $s' = 1$ **then**

return $(\theta^-, r^-), (\theta^+, r^+), \theta', n', s', \alpha', n'_\alpha;$

if $v = -1$ **then**

$(\theta^-, r^-), (-, -), \theta'', n'', s'', \alpha'', n''_\alpha \leftarrow \text{BuildTree}(\theta^-, r^-, u, v, j - 1, \epsilon);$

else

$(-, -), (\theta^+, r^+), \theta'', n'', s'', \alpha'', n''_\alpha \leftarrow \text{BuildTree}(\theta^+, r^+, u, v, j - 1, \epsilon);$

$p \leftarrow \frac{n''}{n' + n''};$

if $\text{Bernoulli}(p)$ **then**

$\theta' \leftarrow \theta'';$

$n' \leftarrow n' + n'';$

$s' \leftarrow s' \cdot s'' \cdot \text{Turning}(\theta^-, r^-, \theta^+, r^+);$

$\alpha' \leftarrow \alpha' + \alpha'';$

$n'_\alpha \leftarrow n'_\alpha + n''_\alpha;$

return $(\theta^-, r^-), (\theta^+, r^+), \theta', n', s', \alpha', n'_\alpha;$

4 Final algorithm

Algorithm 3: No-U-Turn Sampler with Dual Averaging

Given θ_0 , δ , L , M , M^{adapt} :

// **FindReasonableEpsilon**(θ , **key**) $\rightarrow$ **epsilon**

Set $\epsilon_0 = \text{FindReasonableEpsilon}(\theta)$,

$\mu = \log 10\epsilon_0$, $\bar{\epsilon}_0 = 1$, $\bar{H}_0 = 0$, $\gamma = 0.05$, $t_0 = 10$, $k = 0.75$.

Input: Array A di lunghezza n , valore x

Output: Indice i tale che $A[i] = x$, oppure **None**

for $i \leftarrow 1$ **to** n **do**

if $A[i] = x$ **then**
 return i

return **None**
